

PISSA-H v1.2.1 Production Specification (Consolidated)

Status: Normative (MUST/SHOULD/MAY).

Scope: Deterministic, auditable construction and interchange of Hamiltonians and operator sums (primarily Pauli-string-based), as a *tooling/IR* layer (not a physics claim).

0. Goals and non-goals

0.1 Goals

- **Determinism:** identical input specs produce **byte-identical** IR artifacts across runs, machines, and thread counts.
- **Auditability:** exact multiplicity/parity audits plus scoped exact transforms and scalable probabilistic fingerprints.
- **Interoperability:** adapters to/from major quantum software stacks; stable JSON IR for cross-tool communication.
- **Scalability:** support $n > 64$ qubits without silent truncation.

0.2 Non-goals

- No claim of asymptotic speedups for unstructured worst-case Hamiltonians.
 - Not a new simulation algorithm; focuses on IR correctness, reproducible builds, and structured compression.
-

1. Architecture

1.1 Three-layer separation

1. **Foundation layer**
 2. Stable atom registries (deterministic IDs)
 3. Versioned canonical encoding
 4. Collision-resistant hashing with collision safety
 5. **Core (Modules A–C)**
 6. Deterministic canonicalization and merging
 7. Factorized IR (templates + placements) with caching
 8. Tiered audit system
 9. **Interoperability (Module D)**
 10. Import/export adapters for frameworks
 11. Stable JSON IR schema
-

2. Normative requirements

2.1 MUST

1. **Deterministic output:** byte-identical IR for identical input specs, independent of thread count/execution order.
2. **Collision safety:** collisions MUST be detected by **bitwise canonical comparison**; collisions MUST fall back to collision buckets.
3. **Scalable masks:** MUST support $n > 64$ (no silent truncation).
4. **Float normalization:** canonical encodings MUST reject NaN/Inf; MUST normalize $-0.0 \rightarrow +0.0$.
5. **Self-hash exclusion:** hash metadata fields MUST be excluded from bytes being hashed.
6. **Stable AtomIDs:** AtomID assignment MUST be deterministic across runs/machines.
7. **Unambiguous encoding:** hashable bytes MUST be unambiguously delimited (canonical CBOR preferred).
8. **Mask normal form:** bits $\geq n_{\text{qubits}}$ MUST be zero; word length MUST match $\text{ceil}(n/64)$.
9. **Hash domain separation:** KeyHash and other hashes/derived seeds MUST use explicit domain labels.

2.2 SHOULD

1. **Fixed-point coefficients:** deterministic merging SHOULD use integer/fixed-point internally.
2. **Template specialization:** template operators SHOULD support multiple encodings (PauliSumLocal/Diagonal/Sparse/Dense).
3. **Deterministic audits:** audit scheduling SHOULD use deterministic RNG seeds derived from the build spec.

2.3 MAY

1. **Float compatibility mode:** IEEE-754 binary64 MAY be accepted in compatibility mode, but MUST normalize and MAY be quantized into fixed-point for canonical IR.
2. **Parallel reduction:** parallel summation MAY be used if paired with deterministic post-processing (sorted reduction or exact arithmetic).

3. Deterministic build model

3.1 Inputs

- **BuildSpecification** (normative): includes
 - canonicalized model parameters
 - chosen coefficient encoding and **fixed-point global scale**
 - chosen audit policy/version
 - registry namespaces and atom naming rules
 - any template libraries (versioned)

3.2 Build ID

- **BuildID** = $H(\text{domain}=\text{"PISSAH-BuildID-v1"} \parallel \text{canonical}(\text{BuildSpecification}))$
- BuildID MUST be deterministic; non-deterministic metadata (timestamps, hostnames) MUST NOT be inside deterministic artifacts.

3.3 Deterministic metadata

- If a human-facing timestamp/log is desired, it MUST be emitted as a **sidecar report** that is not covered by byte-identical determinism requirements.

4. Foundation data types

4.1 Atom registry (stable IDs)

Rule: Final IR MUST be emitted after the registry is **frozen**.

Algorithm (normative):

1. Collect all atoms as pairs `(namespace, name)`.
2. Convert each to a deterministic byte representation (see §4.1.1).
3. Sort lexicographically by `(namespace_bytes, name_bytes)`.
4. Assign AtomIDs sequentially starting at 1.

4.1.1 String normalization policy (MUST pick one)

- **Option A (simplest):** treat namespace/name as raw UTF-8 bytes; no Unicode normalization.
- **Option B:** normalize to NFC then treat as UTF-8 bytes.

Implementations MUST document which policy they use; all interoperating tools MUST use the same policy.

4.2 QuantumMask (scalable bitmasks)

```
enum QuantumMask {  
    Small { value: u64, n_qubits: u8 }, // 1..=64  
    Big { words: Vec<u64>, n_qubits: usize }, // >64  
}
```

4.2.1 Canonical normal form (MUST)

- `n_words = ceil(n_qubits/64)`.
- Serialization MUST include `n_qubits` and `n_words`.
- The serialized `words.len()` MUST equal `n_words` exactly.
- Bits at indices `i >= n_qubits` MUST be zero.

4.2.2 Canonical bytes (MUST)

Little-endian for integers and for word order:

- Word 0 covers qubits 0–63, word 1 covers 64–127, etc.

```
fn canonical_bytes(&self) -> Vec<u8> {  
    // n_qubits: u64 LE || n_words: u32 LE || words: u64 LE...  
}
```

4.3 CanonicalCoefficient (binary fixed-point preferred)

4.3.1 Binary fixed-point (NORMATIVE for deterministic modes)

```
struct FixedPointCoefficient {  
    real: i128,  
    imag: i128,  
    scale: i8, // value = (real + i*imag) * 2^(-scale)  
}
```

4.3.2 Global scale (MUST)

- The build spec MUST declare the global fixed-point scale.
- Tools MUST reject IRs whose coefficients claim a scale inconsistent with the declared build spec.

4.3.3 Float-to-fixed conversion rounding (MUST)

To ensure cross-language determinism, the conversion MUST specify a rounding rule:

- **RECOMMENDED:** round-to-nearest, ties-to-even.

Implementations MUST use the specified rule exactly.

4.3.4 Float normalization (MUST)

- Reject NaN and $\pm\text{Inf}$.
- Normalize $-0.0 \rightarrow +0.0$ before encoding.

4.4 Canonical encoding and hashing

4.4.1 HashableEncoding (preferred)

All bytes hashed into KeyHash MUST come from a single unambiguous canonical encoding.

```

struct HashableEncoding {
    version_major: u16,
    version_minor: u16,
    namespace_id: u32,
    template_type: AtomID,
    atoms: Vec<AtomID>,
    local_indices: Vec<u8>,
    parameters: Vec<u8>, // canonical CBOR bytes
}

```

4.4.2 Canonical CBOR (preferred)

- Encode HashableEncoding using **RFC 8949 canonical CBOR rules**.
- Maps MUST have keys sorted by bitwise lexicographic order.
- No indefinite-length items.

4.4.3 Parameters embedding (MUST clarify)

- `parameters` MUST be embedded as a **CBOR byte string (bstr)** containing canonical CBOR bytes of the parameter object.

4.4.4 Metadata exclusion (MUST)

- Any metadata fields (self-hash, timestamps, build environment) MUST be excluded from HashableEncoding.

4.4.5 Versioning rule (MUST)

- Canonical encodings include **MAJOR.MINOR only**.
- PATCH versions MUST NOT change canonical bytes.

4.4.6 Domain-separated KeyHash (MUST)

- `KeyHash = BLAKE3("PISSAH-KeyHash-v1" || canonical_cbor(HashableEncoding))`

5. Core IR (Modules A–C)

5.1 Operator representations

- **Compatibility mode (term stream):** Raw terms with (xmask, zmask, coeff).
- **Factorized mode:** Templates + Placements.

5.1.1 Pauli representation

- Use symplectic form: `(xmask, zmask)` with a phase convention defined by the implementation.
- Canonical term identity MUST include the full symplectic masks and coefficient encoding.

5.2 Template operators

5.2.1 TemplateOperator (deterministic modes)

Template operator coefficients MUST be fixed-point (or exact rational).

- DenseMatrix (small k)
- Diagonal
- PauliSumLocal (recommended)
- Sparse

5.2.2 TemplateOperatorCompat (compatibility)

Float-based template operators MAY exist only in compatibility mode and MUST enforce float normalization rules.

6. Deterministic processing rules

6.1 Compatibility modes

- **Mode 0 (compatibility/audit-only):** ingest terms, compute audits, no merging required.
- **Mode 1 (dedup + deterministic ordering):** canonicalize + KeyHash + merge + deterministic output.
- **Mode 2 (factorized):** extract templates, compile once, bind placements; caches enabled.
- **Strict mode:** fail-fast on any non-determinism, invalid floats, unknown atoms after freeze, or hash collision policy violations.

6.2 Deterministic parallel merging

Protocol (normative):

1. Thread-local accumulation into maps keyed by KeyHash.
2. Collect union of keys; sort keys lexicographically by KeyHash bytes.
3. For each key, collect contributions from threads.
4. Sort contributions by deterministic fingerprint.
5. Merge in sorted order using deterministic arithmetic (fixed-point/rational).

6.2.1 Deterministic fingerprint (MUST)

- Fingerprint MUST be canonical bytes of the coefficient (or a domain-separated hash of those bytes).

6.3 Collision handling (MUST)

- When inserting a term:
- if KeyHash absent: insert (store canonical bytes)
- if KeyHash present:
 - compare canonical bytes
 - if equal: merge

- else: place into collision bucket keyed by KeyHash
 - Strict mode MAY fail-fast on collision detection.
-

7. Audit system

7.1 Levels

- **L1 (always-on):** multiplicity, parity, coefficient sums per key.
- **L2 (phase boundaries):** exact subset zeta/Möbius on bounded supports $m \leq m_{\max}$.
- **L3 (scheduled):** probabilistic fingerprints via Walsh–Hadamard sampling / random projections.

7.2 Deterministic audit scheduling (SHOULD; MUST in strict mode)

- RNG MUST be seeded from BuildID with domain separation.
- Sampling schedule MUST be deterministic (phase boundaries, deterministic counters).

7.3 Scalable projections (MUST)

- Projections MUST use QuantumMask (no 128-bit cap).
-

8. Interoperability (Module D)

8.1 JSON IR schema

- JSON IR MUST include:
 - format_version
 - BuildID
 - atom_registry final mapping
 - coefficient encoding (binary fixed-point scale)
 - templates and placements
 - audit trail (optional)

Determinism note: timestamps and runtime metadata MUST NOT be present in deterministic artifacts; if included, they MUST be excluded from byte-identical requirements and clearly marked.

8.2 Framework adapters (MUST)

Adapters MUST implement:

- import_from_framework
- export_to_framework
- validate_roundtrip

8.3 Roundtrip validation tolerance (SHOULD)

Tolerance SHOULD account for fixed-point quantization:

- Either require sufficiently high scale,
 - Or define epsilon as a function of scale, e.g. $\epsilon \geq C * 2^{(-scale)}$.
-

9. Benchmark and compliance suite

9.1 Required baselines

- Naive term generation (no dedup)
- Hash-based dedup only
- Full PISSA-H factorized IR

9.2 Required families (minimum)

- 2D Heisenberg (structured/high repetition)
- QAOA MaxCut on regular graphs (medium repetition)
- Random sparse k-local (low repetition)

9.3 Determinism tests (MUST)

- 100 runs, varying threads, byte-identical output.

9.4 Fault injection tests (MUST)

- duplicates detected by L1 multiplicity
 - cancelling pairs detected by parity
 - random corruption detected by L3 with specified probability target
-

10. Security and reliability

10.1 Cryptographic assumptions

- BLAKE3 assumed collision-resistant for practical purposes.
- ChaCha-family RNG used for reproducible sampling (seeded deterministically).

10.2 Failure modes

- Hash collisions → collision buckets (and strict-mode failure option)
 - Memory corruption → checksum/self-hash validation, reject corrupted entries
 - Float nondeterminism → fixed-point canonicalization; reject invalid floats
 - Registry mismatch → require frozen registry mapping embedded in IR
-

11. External references (bibliography)

Standards / specs

- RFC 8949: *Concise Binary Object Representation (CBOR)*.
- RFC 8439: *ChaCha20 and Poly1305 for IETF Protocols*.
- BLAKE3 specification (BLAKE3-team): *The BLAKE3 paper: specifications, analysis, and design rationale*.
- IETF Internet-Draft: *The BLAKE3 Hashing Framework* (draft).

Subset transforms / Möbius / WHT

- Björklund et al.: *Fourier Meets Möbius: Fast Subset Convolution*.
- Björklund (SIAM): *Fast Zeta Transforms for Lattices with Few Irreducibles*.
- Kaski et al.: *Fast Möbius inversion in semimodular lattices*.

Stabilizers / symplectic Pauli representations

- Aaronson & Gottesman: *Improved Simulation of Stabilizer Circuits* (Phys. Rev. A 70, 052328).

Framework interop anchors

- Qiskit: `SparsePauliOp` API and operator representation docs.
- Cirq: `PauliString` / `PauliSum` reference docs.
- OpenQASM 3 specification.

Reference links (as code block)

```
CBOR (RFC 8949): https://www.rfc-editor.org/rfc/rfc8949.html
ChaCha20-Poly1305 (RFC 8439): https://www.rfc-editor.org/rfc/rfc8439.html
BLAKE3 spec PDF: https://raw.githubusercontent.com/BLAKE3-team/BLAKE3-specs/master/blake3.pdf
BLAKE3 IETF draft: https://www.ietf.org/archive/id/draft-aumasson-blake3-00.html
Aaronson & Gottesman (PhysRevA): https://doi.org/10.1103/PhysRevA.70.052328
Qiskit SparsePauliOp API: https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.quantum_info.SparsePauliOp
Cirq PauliSum: https://quantumai.google/reference/python/cirq/PauliSum
Cirq PauliString: https://quantumai.google/reference/python/cirq/PauliString
OpenQASM 3: https://openqasm.com/versions/3.0/index.html
Subset convolution slides: https://people.csail.mit.edu/rrw/presentations/subset-conv.pdf
Fast zeta transforms (SIAM): https://epubs.siam.org/doi/pdf/10.1137/1.9781611973099.113
```